

Linear Systems

Digital Signal Processing

May 21, 2026

Requirements for Linearity: Homogeneity, Additivity & Shift Invariance

A system $\mathcal{T}$ is **linear** iff it satisfies *both*:

1. Homogeneity (Scaling)

If $x[n] \xrightarrow{\mathcal{T}} y[n]$, then $\alpha x[n] \xrightarrow{\mathcal{T}} \alpha y[n]$, $\forall \alpha \in \mathbb{C}$

2. Additivity

If $x_1 \xrightarrow{\mathcal{T}} y_1$ and $x_2 \xrightarrow{\mathcal{T}} y_2$, then $x_1[n] + x_2[n] \xrightarrow{\mathcal{T}} y_1[n] + y_2[n]$

Combined: Superposition Principle

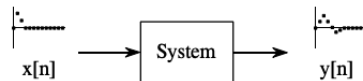
$$\mathcal{T}\{\alpha_1 x_1[n] + \alpha_2 x_2[n]\} = \alpha_1 y_1[n] + \alpha_2 y_2[n]$$

3. Shift (Time) Invariance

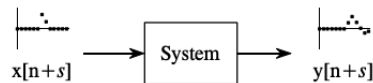
$x[n - n_0] \xrightarrow{\mathcal{T}} y[n - n_0] \forall n_0 \in \mathbb{Z}$

Behaviour **unchanged** with time origin. Test: compute $\mathcal{T}\{x[n - n_0]\}$ and $y[n - n_0]$ — must be identical.

IF



THEN



Definition of shift invariance. A system is said to be shift invariant if a shift in the input signal causes an identical shift in the output signal. In mathematical terms, if $x[n]$ produces $y[n]$, then $x[n+s]$ produces $y[n+s]$, for any signal, $x[n]$, and any constant, s .

Static Linearity & Sinusoidal Fidelity

Static & Sinusoidal Linearity

Static Linearity: For constant (DC) inputs, output is proportional to input:

$$y[n] = a x[n], \quad a \in \mathbb{R}$$

I/O characteristic is a **straight line through the origin**.

Examples: **Ohm's Law** and **Hooke's Law**.

Nonlinear examples: diode characteristic, magnetic hysteresis.

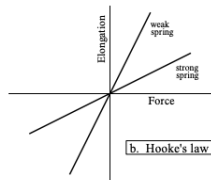
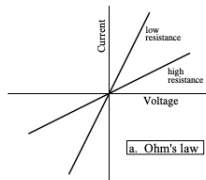
Memoryless System: Output depends only on the present input sample, not past/future values. If a memoryless system has static linearity, then it is linear.

Sinusoidal Fidelity: A sinusoidal input produces a sinusoidal output at the *same frequency*:

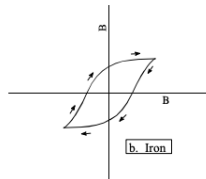
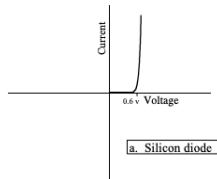
$$x[n] = A \sin(\omega n + \phi) \Rightarrow y[n] = B \sin(\omega n + \theta)$$

Amplitude and phase may change, but frequency remains unchanged.

Static Linearity



DC nonlinearity



Special Properties of Linearity

Commutative Property of Convolution

For LTI systems in series, **order does not matter**:

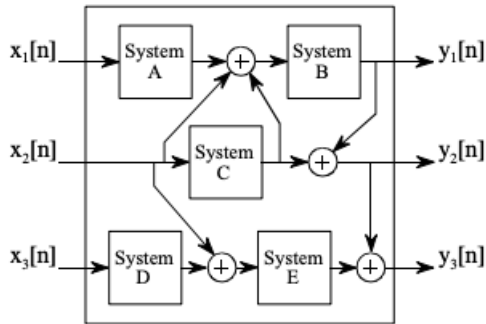
$$x[n] * h[n] = h[n] * x[n] \quad h_1 * h_2 = h_2 * h_1$$

All Three Convolution Properties

Property	Relation
Commutative	$x * h = h * x$
Associative	$(x * h_1) * h_2 = x * (h_1 * h_2)$
Distributive	$x * (h_1 + h_2) = x * h_1 + x * h_2$

Practical Significance

Filters can be **reordered in a cascade** without changing the overall frequency response. Critical for filter design & hardware implementation.



Any system with multiple inputs and/or outputs will be linear if it is composed of linear systems and signal additions.

Synthesis vs. Decomposition

Synthesis

- Combine signals via *scaling* (multiply by a constant) then *addition*
- Signals *cannot* be multiplied by each other in a linear system
- Example: $x[n] = a_0x_0[n] + a_1x_1[n] + a_2x_2[n]$

Decomposition

- Inverse of synthesis: split one signal into additive components
- *Infinitely* many valid decompositions exist
- Example: $40 = 1 + 39 = 2 + 38 = \dots$

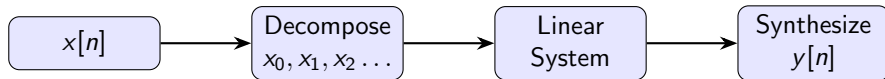
Key Constraint in Linear Systems

The **only** allowed operations are scaling and addition — no signal-to-signal multiplication.

Superposition — The Heart of DSP

Principle

Pass a complex signal $x[n]$ through a linear system by decomposing it, processing each simple component, then synthesizing the outputs.



- Each component $x_k[n]$ produces output $y_k[n]$ independently
- Final output: $y[n] = y_0[n] + y_1[n] + y_2[n] + \dots$
- **Result is identical** to passing $x[n]$ directly through the system

Why Superposition Matters

Mental Math Analogy

2041×4 can be decomposed as:

$$(2000 + 40 + 1) \times 4 = 8000 + 160 + 4 = \mathbf{8164}$$

Superposition turns one hard problem into several easy ones.

Three take-aways for DSP:

- 1 Instead of analysing a *complex* signal, understand how *simple* signals behave.
- 2 Signals and systems are viewed as a **superposition (sum) of simpler waveforms**.
- 3 This is the foundation of nearly **all** signal-processing techniques (Fourier, filtering, convolution, ...).

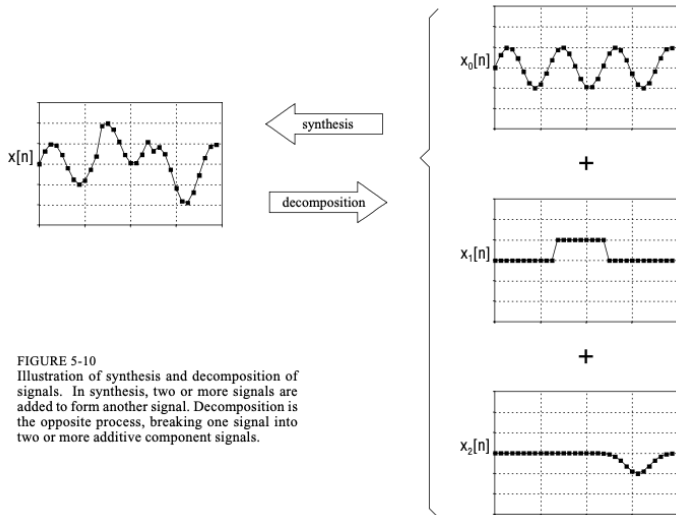


FIGURE 5-10
Illustration of synthesis and decomposition of signals. In synthesis, two or more signals are added to form another signal. Decomposition is the opposite process, breaking one signal into two or more additive component signals.

Even–Odd Decomposition: Definition

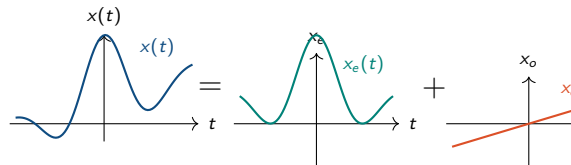
Definition

Every signal $x(t)$ can be **uniquely** written as

$$x(t) = x_e(t) + x_o(t)$$

where

$$x_e(t) = \frac{x(t) + x(-t)}{2}, \quad x_o(t) = \frac{x(t) - x(-t)}{2}$$



A signal (blue) split into its even (teal) and odd (red) parts.

Key Properties

Even part: $x_e(-t) = x_e(t)$

Odd part: $x_o(-t) = -x_o(t)$

Verify: $x_e(t) + x_o(t) = x(t) \checkmark$

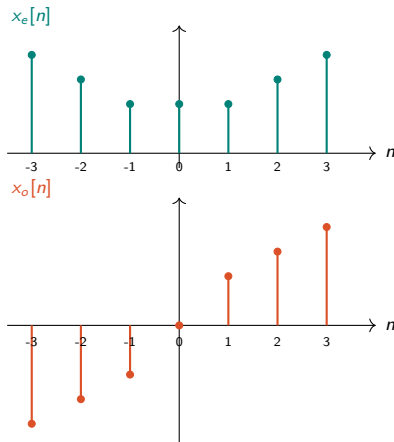
Even-Odd Decomposition: Discrete-Time Signals

DT Formulas

$$x_e[n] = \frac{x[n] + x[-n]}{2}, \quad x_o[n] = \frac{x[n] - x[-n]}{2}$$

Example: $x[n] = \{1, 2, 3, 4\}$ for $n = 0, 1, 2, 3$, zero otherwise.

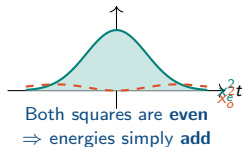
n	$x[n]$	$x_e[n]$	$x_o[n]$
-3	0	2	-2
-2	0	$3/2$	$-3/2$
-1	0	1	-1
0	1	1	0
1	2	1	1
2	3	$3/2$	$3/2$
3	4	2	2



Even–Odd Decomposition: Energy Orthogonality

Energy Separation Theorem

$$E_x = \int_{-\infty}^{\infty} |x(t)|^2 dt = \underbrace{\int_{-\infty}^{\infty} x_e^2(t) dt}_{E_{x_e}} + \underbrace{\int_{-\infty}^{\infty} x_o^2(t) dt}_{E_{x_o}}$$



DT version

$$\sum_{n=-\infty}^{\infty} |x[n]|^2 = \sum_n x_e^2[n] + \sum_n x_o^2[n]$$

Proof sketch:

$$\begin{aligned} \int x^2 &= \int (x_e + x_o)^2 \\ &= \int x_e^2 + 2 \underbrace{\int x_e x_o}_{=0} + \int x_o^2 \end{aligned}$$

The cross-term vanishes because $x_e x_o$ is odd.

Interlaced Decomposition

Core Idea

Split $x[n]$ into **even-** and **odd-indexed** subsequences, then *interlace* (interleave) them back like zipper teeth.

Mathematical Representation

$$x[n] = \begin{cases} x_e[n/2], & n \text{ even} \\ x_o[(n-1)/2], & n \text{ odd} \end{cases}$$

- $x_e[m] = x[2m]$ (even picks)
- $x_o[m] = x[2m+1]$ (odd picks)

Basic Example

$$x[n] = [1, 2, 3, 4, 5, 6]$$

$$\text{Even } x_e: [1, 3, 5]$$

$$\text{Odd } x_o: [2, 4, 6]$$

Worked Example

$$x[n] = [4, 7, 2, 9, 5, 1]$$

Index	n	Value
$x_e[0]$	$n = 0$	4
$x_o[0]$	$n = 1$	7
$x_e[1]$	$n = 2$	2
$x_o[1]$	$n = 3$	9
$x_e[2]$	$n = 4$	5
$x_o[2]$	$n = 5$	1

$$\Rightarrow x_e = [4, 2, 5], \quad x_o = [7, 9, 1]$$

Impulse Decomposition

Impulse Decomposition

A signal $\{1, 3, 2, 5\}$ is broken into scaled impulses — each sample becomes its own signal with zeros elsewhere:

$$\{1, 3, 2, 5\} \longrightarrow \begin{cases} \{1, 0, 0, 0\} & \text{sample at } n = 0 \\ \{0, 3, 0, 0\} & \text{sample at } n = 1 \\ \{0, 0, 2, 0\} & \text{sample at } n = 2 \\ \{0, 0, 0, 5\} & \text{sample at } n = 3 \end{cases}$$

Step Decomposition

Step Decomposition

The same signal is broken into scaled *step functions* — each component starts at a sample and holds a constant value from that point on. Each step equals the *difference* from the previous sample:

$$\{1, 3, 2, 5\} \longrightarrow \begin{cases} \{1, 1, 1, 1\} & \Delta_0 = 1 \\ \{0, 2, 2, 2\} & \Delta_1 = 3 - 1 = 2 \\ \{0, 0, -1, -1\} & \Delta_2 = 2 - 3 = -1 \\ \{0, 0, 0, 3\} & \Delta_3 = 5 - 2 = 3 \end{cases}$$

In both cases, summing all component signals recovers the original $\{1, 3, 2, 5\}$.